



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO NACIONAL DE MÉXICO INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:

Agosto - Diciembre 2025

CARRERA:

Ingeniería en Sistemas Computacionales

MATERIA:

Patrones de Diseño

TÍTULO ACTIVIDAD:

Examen U2

Integrantes:

Antunez Partida Jorge Octavio - 21211910

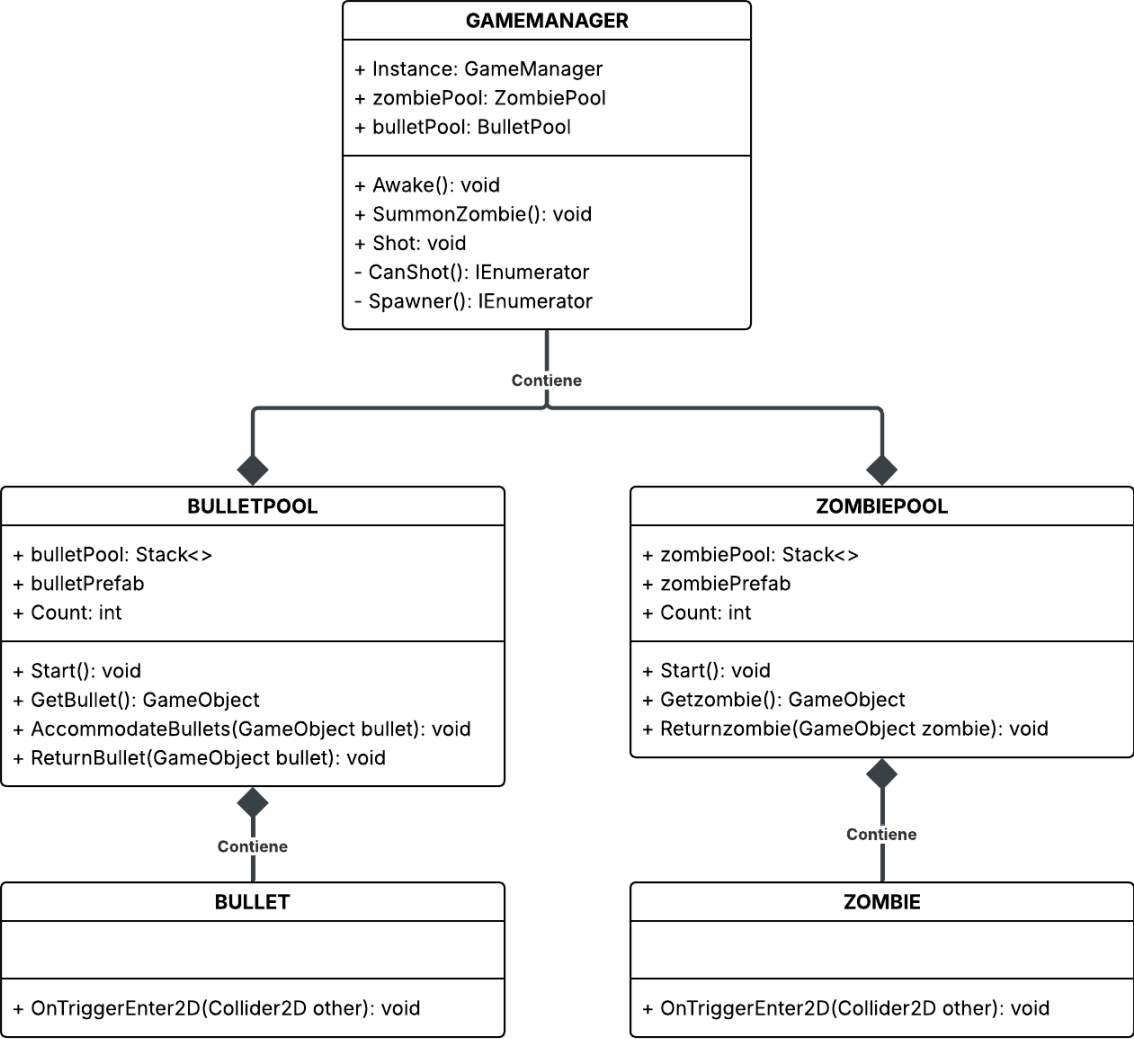
NOMBRE DEL MAESTRO(A):

Maribel Guerrero Luis

Fecha

17/10/2025

UML



Código

Clase GameManager que utiliza el patrón singleton

```
6      Script de Unity (1 referencia de recurso) | 11 referencias
7      public class GameManager : MonoBehaviour
8      {
9          13 referencias
10         public static GameManager Instance { get; private set; }
11
12         public Transform Playe;
13
14         public Transform objectPoolMenu;
15         public Transform zombieSpawner;
16
17         public ZombiePool zombiePool;
18         public BulletPool bulletPool;
19
20         public float bulletSpeed = 10.0f;
21         public float zombieZspeed = 3.5f;
22
23         public InputActionAsset InputAction;
24         private InputAction jumpAction;
25
26         public bool canShot = true;
27         public float Shotcooldown = 1;
28
29         public float spawnCooldown = 3;
30
31         public int cantidad = 0;
32
33         Mensaje de Unity | 0 referencias
34         void Awake()
35         {
36             jumpAction = InputAction.FindActionMap("Player").FindAction("Jump");
37
38             if (Instance != null && Instance != this)
39             {
40                 Destroy(gameObject);
41                 return;
42             }
43
44             Instance = this;
45         }
46     }
```

```

44  void Start()
45  {
46      Debug.Log("Juego Empezo");
47      StartCoroutine(Spawner());
48  }
49
50  Mensaje de Unity | 0 referencias
51  void Update()
52  {
53      if (jumpAction.IsPressed() && canShot)
54      {
55          StartCoroutine(CanShot());
56          Shoot();
57      }
58
59      cantidad = bulletPool.Count;
60  }
61
62  1 referencia
63  void SummonZombie()
64  {
65      GameObject zombie = zombiePool.GetZombie();
66      if (zombie == null)
67      {
68          return;
69      }
70
71      float randomY = Random.Range(-2.5f, 2.5f);
72      Vector3 spawnPosition = new Vector3(
73          zombieSpawner.position.x,
74          zombieSpawner.position.y + randomY,
75          zombieSpawner.position.z
76      );
77
78      zombie.transform.position = spawnPosition;
79
80      Rigidbody2D rb = zombie.GetComponent<Rigidbody2D>();
81      rb.linearVelocityX = -zombieZpeed;

```

```

82 1 referencia
83 void Shoot()
84 {
85     GameObject bullet = bulletPool.GetBullet();
86     if (bullet == null)
87     {
88         Debug.LogWarning("No hay balas disponibles para disparar.");
89         return;
90     }
91     bullet.transform.position = Playe.position;
92     Rigidbody2D rb = bullet.GetComponent<Rigidbody2D>();
93     rb.linearVelocityX = bulletSpeed;
94 }
95
96 1 referencia
97 IEnumerator CanShot()
98 {
99     canShot = false;
100     yield return new WaitForSeconds(Shotcooldown);
101     canShot = true;
102 }
103
104 1 referencia
105 IEnumerator Spawner()
106 {
107     while (true)
108     {
109         Debug.Log("Genere Zombie");
110         SummonZombie();
111         yield return new WaitForSeconds(spawnCooldown);
112     }

```

Clase BulletPool que maneja la lógica del patrón ObjectPool de las balas

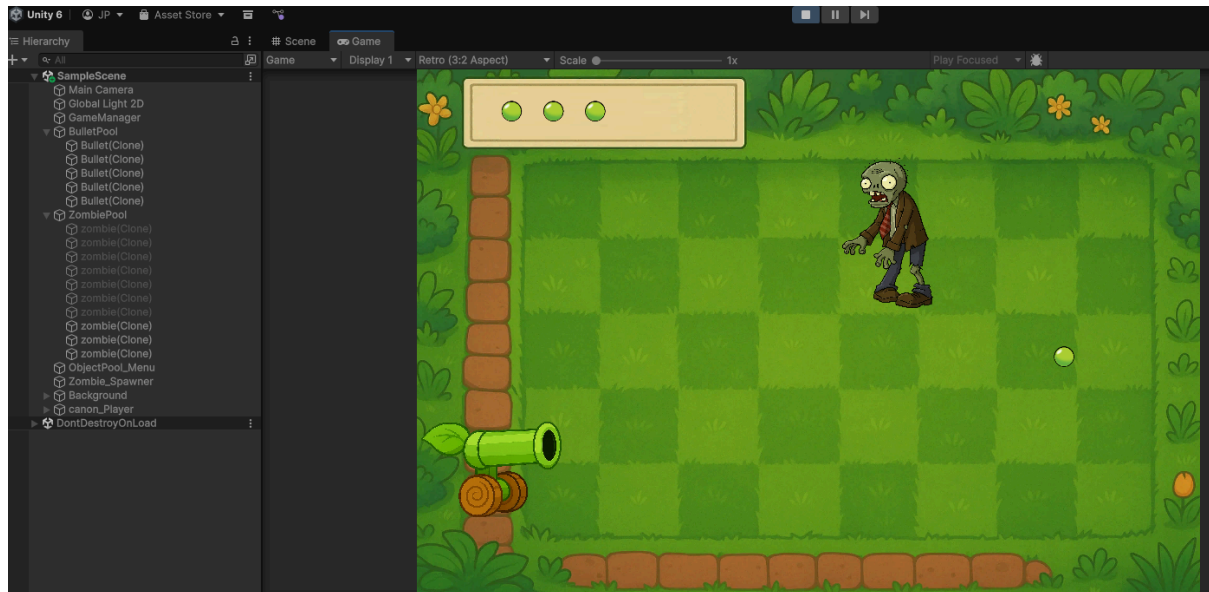
```
6      Script de Unity (1 referencia de recurso) | 1 referencia
7      public class BulletPool : MonoBehaviour
8      {
9
10         public GameObject bulletPrefab;
11
12         public Stack<GameObject> bulletPool = new Stack<GameObject>();
13
14         public int stackCount = 10;
15
16         1 referencia
17         public int Count
18         {
19             get { return bulletPool.Count; }
20         }
21
22         Mensaje de Unity | 0 referencias
23         void Start()
24         {
25             for (int i = 0; i < stackCount; i++)
26             {
27                 GameObject bullet = Instantiate(bulletPrefab);
28                 bullet.transform.parent = transform;
29                 AccomodateBullets(bullet);
30                 bulletPool.Push(bullet);
31             }
32         }
33
34         1 referencia
35         public GameObject GetBullet()
36         {
37             if (bulletPool.Count > 0)
38             {
39                 GameObject bullut = bulletPool.Pop();
40                 bullut.SetActive(true);
41                 return bullut;
42             }
43             else
44             {
45                 return null;
46             }
47         }
48     }
```

```
44 2 referencias
45 public void AccommodateBullets(GameObject bullet)
46 {
47     Rigidbody2D rb = bullet.GetComponent<Rigidbody2D>();
48     rb.linearVelocity = Vector2.zero;
49
50     Vector3 basePos = GameManager.Instance.objectPoolMenu.transform.position;
51
52     int index = bulletPool.Count;
53     float offsetX = 0.8f;
54
55     bullet.transform.position = basePos + Vector3.right * offsetX * index;
56 }
57
58 2 referencias
59 public void ReturnBullet(GameObject bullet)
60 {
61     AccommodateBullets(bullet);
62     bulletPool.Push(bullet);
63 }
```

Clase ZombiePool que maneja la lógica del patrón ObjectPool de los zombies

```
4      Script de Unity (1 referencia de recurso) | 1 referencia
5      public class ZombiePool : MonoBehaviour
6      {
7          public GameObject zombiePrefab;
8
9          public Stack<GameObject> zombiePool = new Stack<GameObject>();
10
11         public int stackCount = 10;
12
13         0 referencias
14         public int Count
15         {
16             get { return zombiePool.Count; }
17         }
18
19         Mensaje de Unity | 0 referencias
20         void Start()
21         {
22             for (int i = 0; i < stackCount; i++)
23             {
24                 GameObject zombie = Instantiate(zombiePrefab);
25                 zombie.transform.parent = transform;
26                 zombie.SetActive(false);
27                 zombiePool.Push(zombie);
28             }
29
30             1 referencia
31             public GameObject GetZombie()
32             {
33                 if (zombiePool.Count > 0)
34                 {
35                     GameObject zombie = zombiePool.Pop();
36                     zombie.SetActive(true);
37                     return zombie;
38                 }
39                 else
40                 {
41                     return null;
42                 }
43             }
44
45             1 referencia
46             public void ReturnZombie(GameObject zombie)
47             {
48                 zombie.SetActive(false);
49                 zombiePool.Push(zombie);
50             }
51         }
```


Ejecución



Conclusión

Estos patrones son muy interesantes y especialmente en el tema de los videojuegos son muy útiles a mi parecer, empezando por el patrón singleton, me resultó muy fácil manejar los elementos del juego como las piscinas de objetos, y acceder a funcionalidades del código principal desde scripts independientes al del manejador del juego.

Por otro lado las piscinas de objetos fueron algo complicadas de implementar pero valen bastante la pena, ya que eliminan proceso como es el instanciar y destruir constantemente objetos, lo cual puede ser pesado para nuestro procesador.

Como conclusión final gracias a este trabajo pude comprender de mucho mejor manera como funciona y la utilidad de estos dos patrones.

Recalcando buenas prácticas, optimizamos el uso de memoria y el rendimiento en ejecución de nuestro programa y evitamos tener varias instancias con valores distintos de algo que debería ser único como lo sería un manejador.